UNIVERSITE D'ABOMEY CALAVI

Institut de Formation en Recherche Informatique

RAPPORT FINAL DE PROJET

Système de Gestion des Actes Administratifs à IFRI

Noms des membres du groupes
DANGBE Imelda
HOUNSOUNON Cléanthe
YAROU Yazid

Enseignant	**Dr TABA CHABI K Wahabou**
Module	**Génie Logiciel**
Groupe	**Groupe 7**
Version logiciel	**1.0.0**
Date	**Mars 2026**

Table des matières

1. Présentation du projet

2. Phase 2 — Modélisation UML
    2.1 Diagramme de cas d'utilisation
    2.2 Diagramme de classes
    2.3 Diagrammes de séquence
        Séquence 1 : Créer une demande (Usager)
        Séquence 2 : Valider/Rejeter une demande (Agent)
        Séquence 3 : Rechercher des actes (Agent)
        Séquence 4 : Consulter les actes (Usager)
    2.4 Diagramme d'activités
3. Phase 3 — Conception architecturale
    3.1 Architecture du système (MVC étendu)
    3.2 Diagramme de composants
    3.3 Diagramme de déploiement
    3.4 Organisation des packages Java
    3.5 Interfaces DAO
    3.6 Technologies utilisées
4. Plan de test

5. Conclusion

1. Présentation du projet

Ce rapport présente l'ensemble des travaux réalisés par le Groupe 7 dans le cadre du Projet 1 de Génie Logiciel (Dr TABA CHABI K Wahabou, UAC-IFRI, 2025-2026). Il regroupe les livrables des Phases 2 et 3 du cycle de développement du logiciel de

gestion des actes administratifs à IFRI.

Objectif du système

Le système vise à automatiser la gestion des actes administratifs à IFRI : enregistrement des demandes, validation, délivrance des actes, archivage et génération de rapports statistiques.

Acteurs du système

Acteur	Rôle
Administrateur	Gère les comptes utilisateurs, configure le système, supervise les opérations et consulte les statistiques.
Agent administratif	Traite les demandes (validation/rejet), génère les actes, effectue les recherches.
Usager	Soumet des demandes d'actes, consulte leur état et télécharge les actes délivrés.

Technologies utilisées

Technologie	Rôle
Java / JavaFX 21	Langage et framework IHM (vues FXML)
Hibernate 6.3	ORM pour la persistance JPA vers MySQL
MySQL 8.x	Base de données relationnelle
Lombok 1.18	Génération automatique du code boilerplate
iText 7	Génération des actes en PDF
Maven	Gestion des dépendances et build

2. Phase 2 — Modélisation UML

La Phase 2 consiste à modéliser le système à l'aide du langage UML. Elle produit les diagrammes statiques et dynamiques nécessaires à la compréhension et à l'implémentation du système.

2.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation identifie les fonctionnalités du système du point de vue de chaque acteur. L'Administrateur supervise et configure, l'Agent traite les demandes et génère les actes, l'Usager soumet et consulte ses demandes.

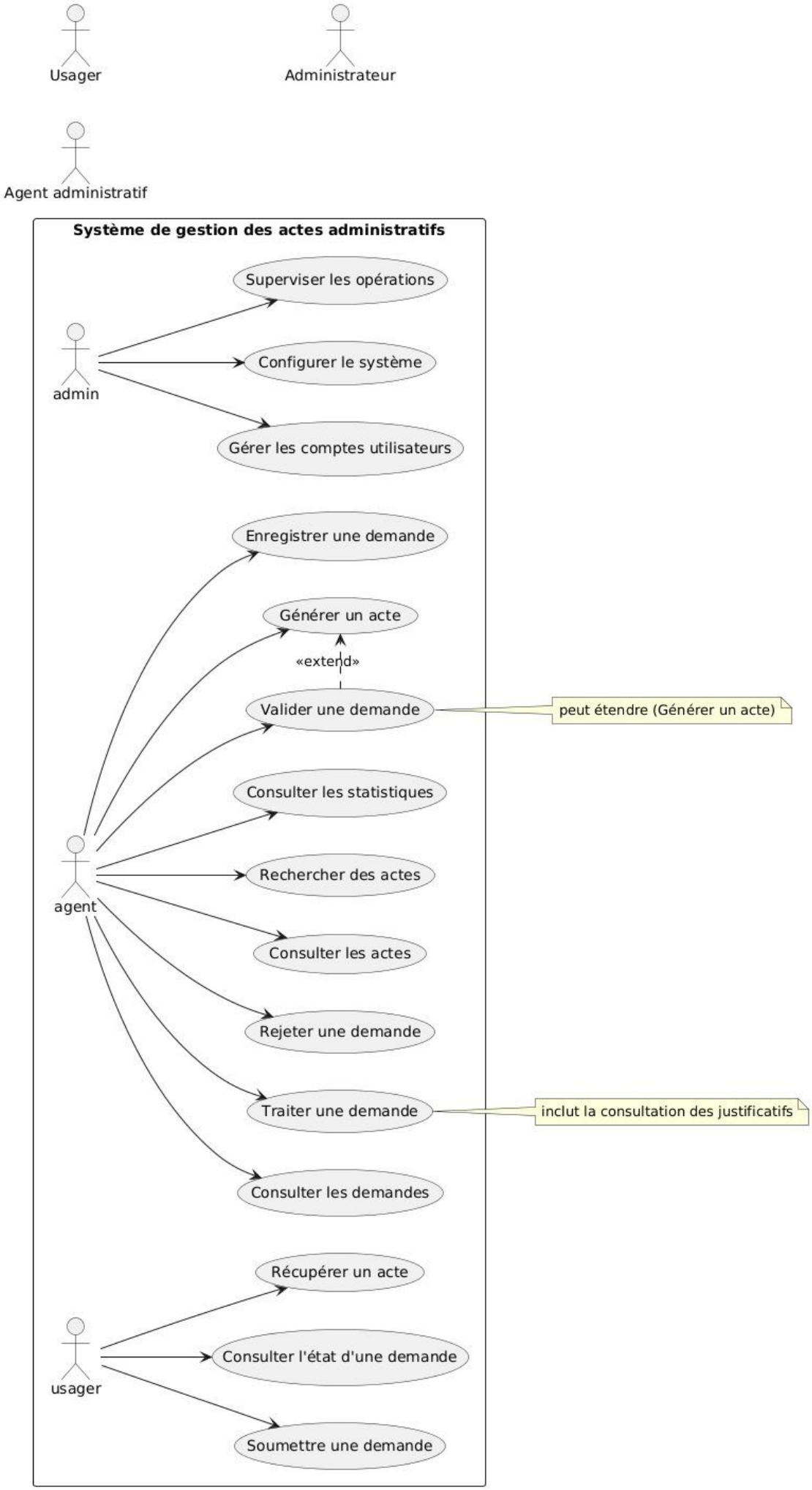
Usager
Administrateur
Agent administratif
Système de gestion des actes administratifs
admin
Superviser les opérations
Configurer le système
Gérer les comptes utilisateurs
agent
Enregistrer une demande
Générer un acte
«extend»
Valider une demande
peut étendre (Générer un acte)
Consulter les statistiques
Rechercher des actes
Consulter les actes
Rejeter une demande
Traiter une demande
inclut la consultation des justificatifs
Consulter les demandes
usager
Récupérer un acte
Consulter l'état d'une demande
Soumettre une demande

Figure 1 — Diagramme de cas d'utilisation

2.2 Diagramme de classes

Le diagramme de classes présente les entités du domaine métier, leurs attributs et les relations entre elles. La classe abstraite Personne est la classe mère de Usager, AgentAdministratif et Administrateur.

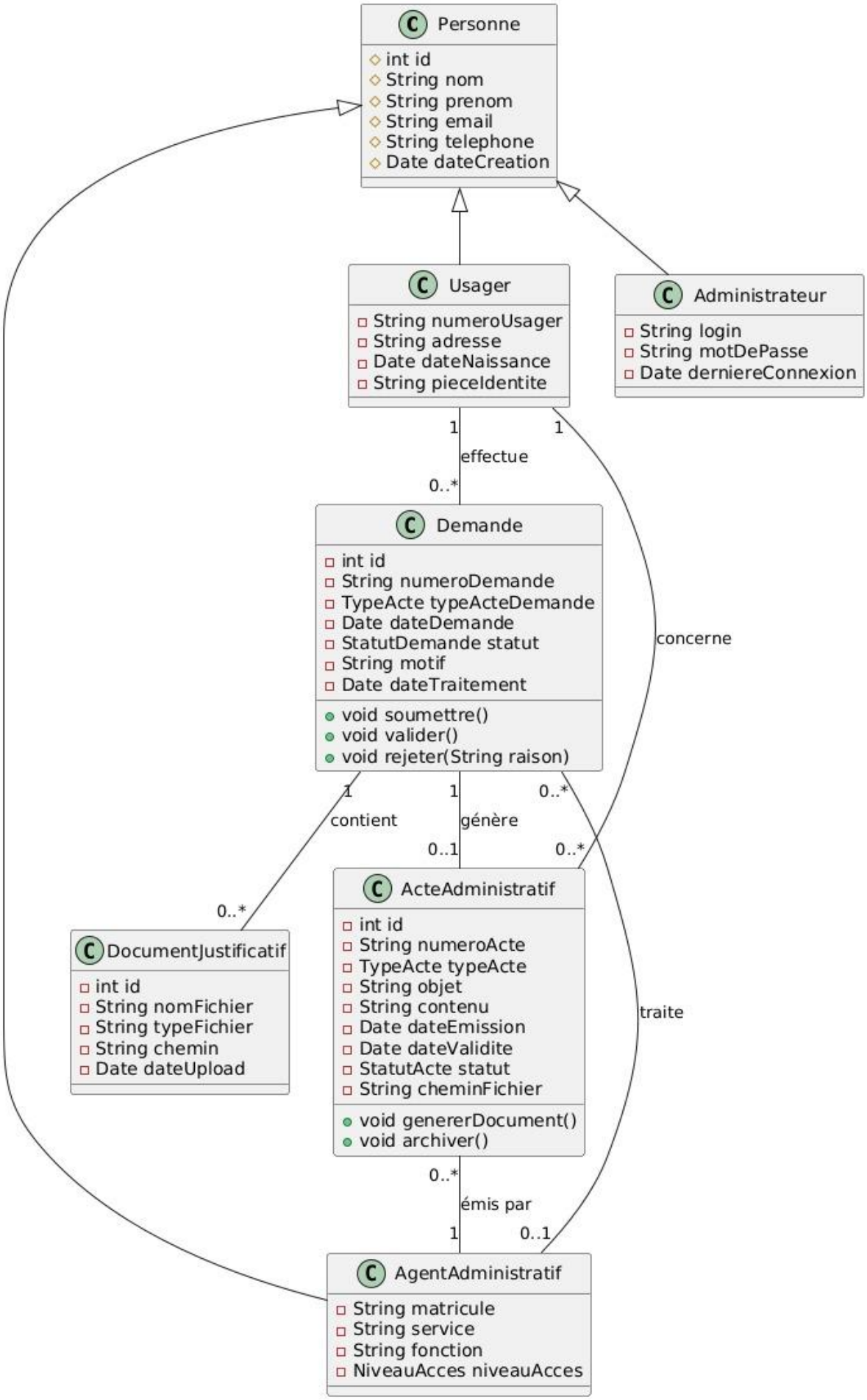
Personne
int id
String nom
String prenom
String email
String telephone
Date dateCreation
Usager
String numeroUsager
String adresse
Date dateNaissance
String pieceIdentite
Administrateur
String login
String motDePasse
Date derniereConnexion
1
effectue
0..*
1
concerne
Demande
int id
String numeroDemande
TypeActe typeActeDemande
Date dateDemande
StatutDemande statut
String motif
Date dateTraitement
void soumettre()
void valider()
void rejeter(String raison)
1
contient
0..*
1
génère
0..1
0..*
0..*
traite
0..1
DocumentJustificatif
int id
String nomFichier
String typeFichier
String chemin
Date dateUpload
ActeAdministratif
int id
String numeroActe
TypeActe typeActe
String objet
String contenu
Date dateEmission
Date dateValidite
StatutActe statut
String cheminFichier
void genererDocument()
void archiver()
0..*
émis par
1
AgentAdministratif
String matricule
String service
String fonction
NiveauAcces niveauAcces

Figure 2 — Diagramme de classes

2.3 Diagrammes de séquence

Les diagrammes de séquence illustrent les interactions entre les acteurs et les composants du système pour les scénarios clés. Quatre scénarios sont présentés.

Séquence 1 : Créer une demande (Usager)

L'usager remplit le formulaire et joint ses justificatifs. Le Controller appelle le Service qui valide les données, sauvegarde la demande en base via le DAO, puis sauvegarde les documents justificatifs.

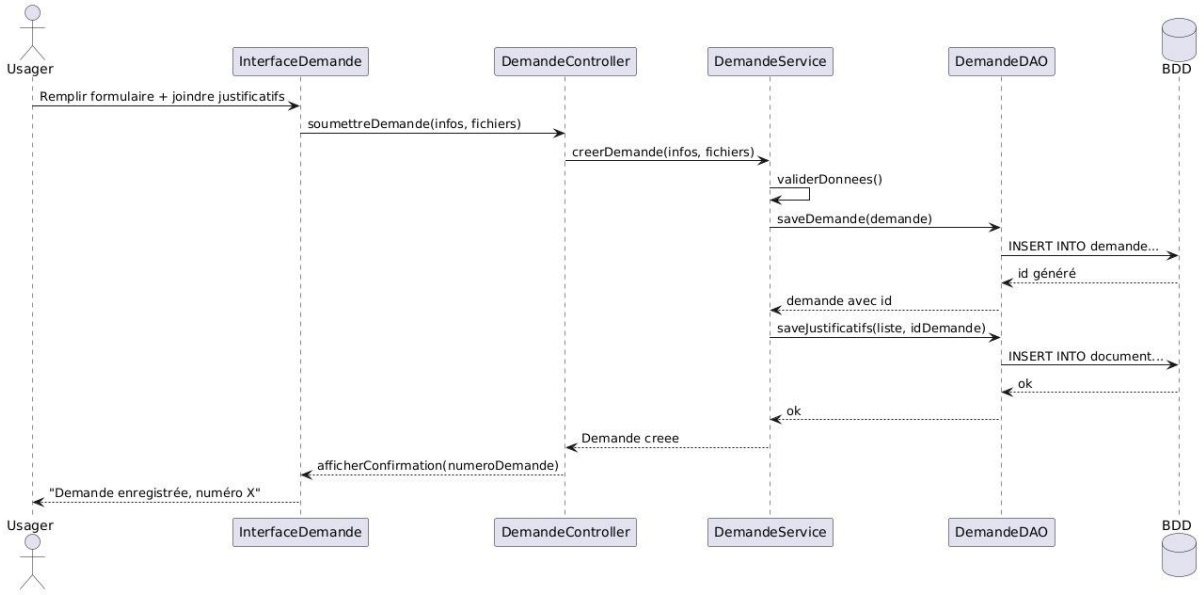

Figure 3 — Séquence : Créer une demande

Séquence 2 : Valider / Rejeter une demande (Agent)

L'agent consulte les demandes en attente, examine les justificatifs et prend une décision. En cas de validation, un acte est automatiquement généré. En cas de rejet, le motif est enregistré.

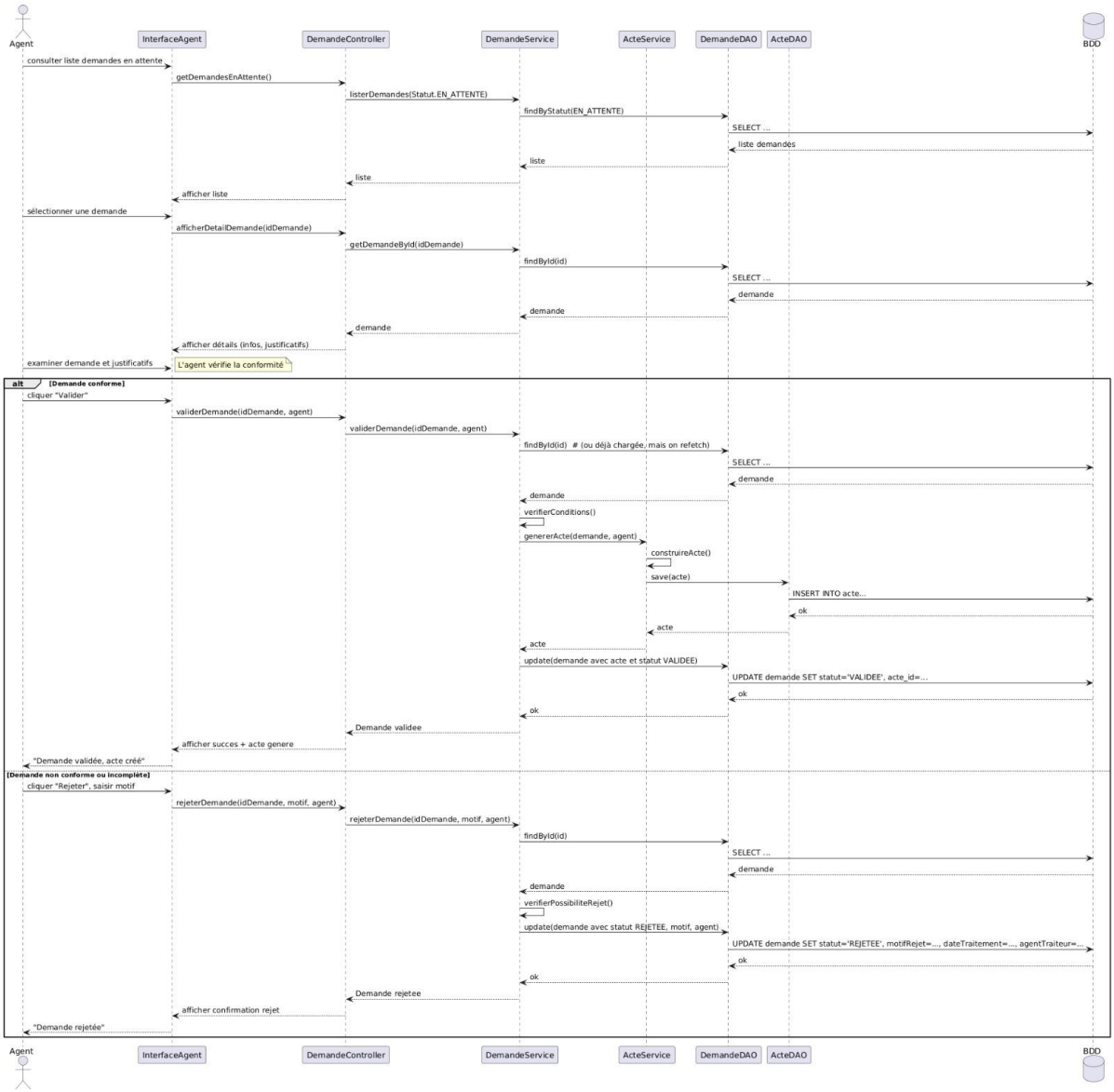

Figure 4 — Séquence : Valider/Rejeter une demande

Séquence 3 : Rechercher des actes (Agent)

L'agent saisit des critères de recherche (type, date, usager). Le Controller appelle ActeService qui interroge ActeDAO pour retourner la liste filtrée.

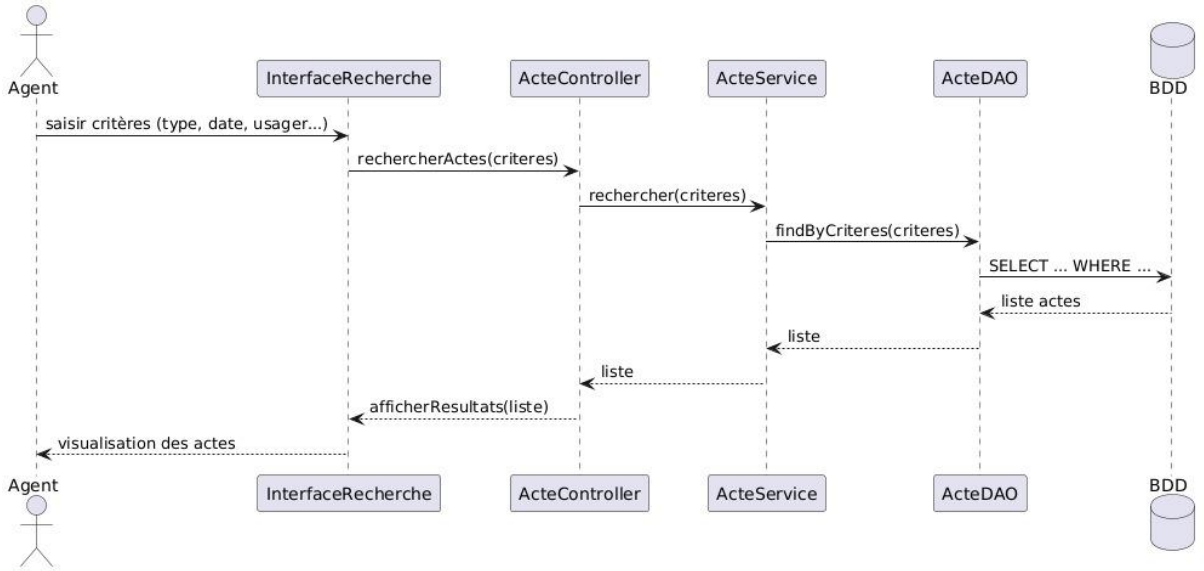

Figure 5 — Séquence : Rechercher des actes

Séquence 4 : Consulter et télécharger un acte (Usager)

L'usager demande la liste de ses actes. Après sélection, le système vérifie ses droits d'accès, lit le fichier depuis le disque et lui retourne le document.

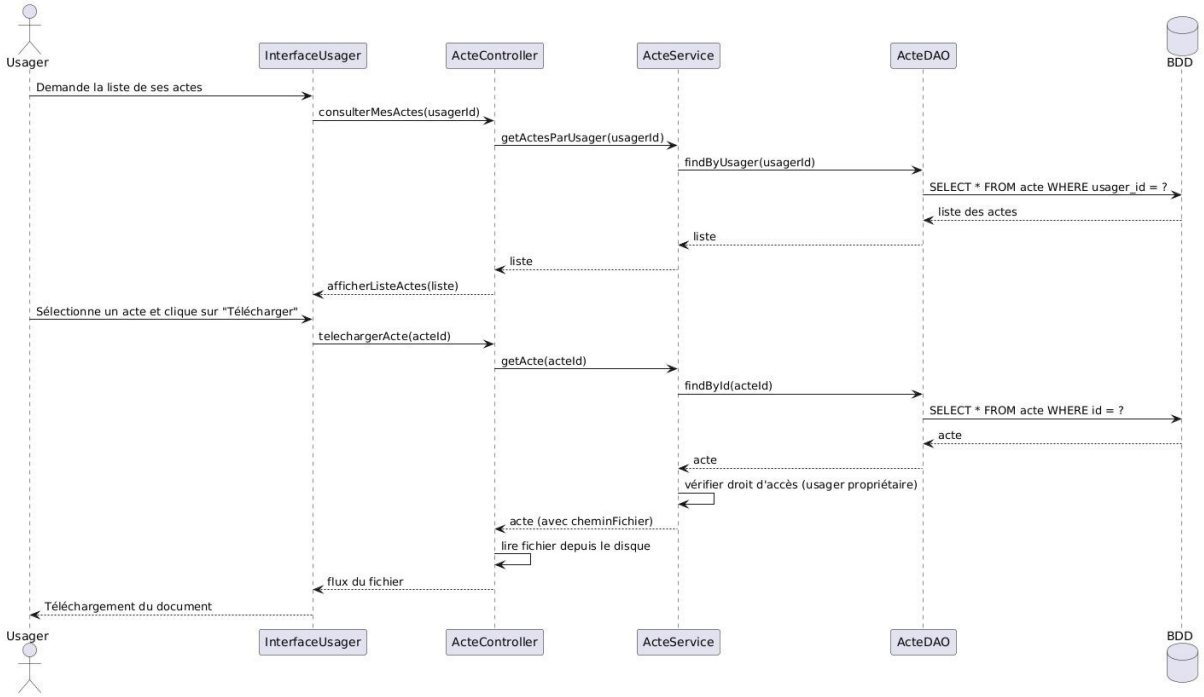

Figure 6 — Séquence : Consulter et télécharger un acte

2.4 Diagramme d'activités

Le diagramme d'activités modélise le flux complet du traitement d'une demande administrative, de la soumission par l'usager jusqu'à l'archivage du dossier, en passant par les décisions de l'agent.

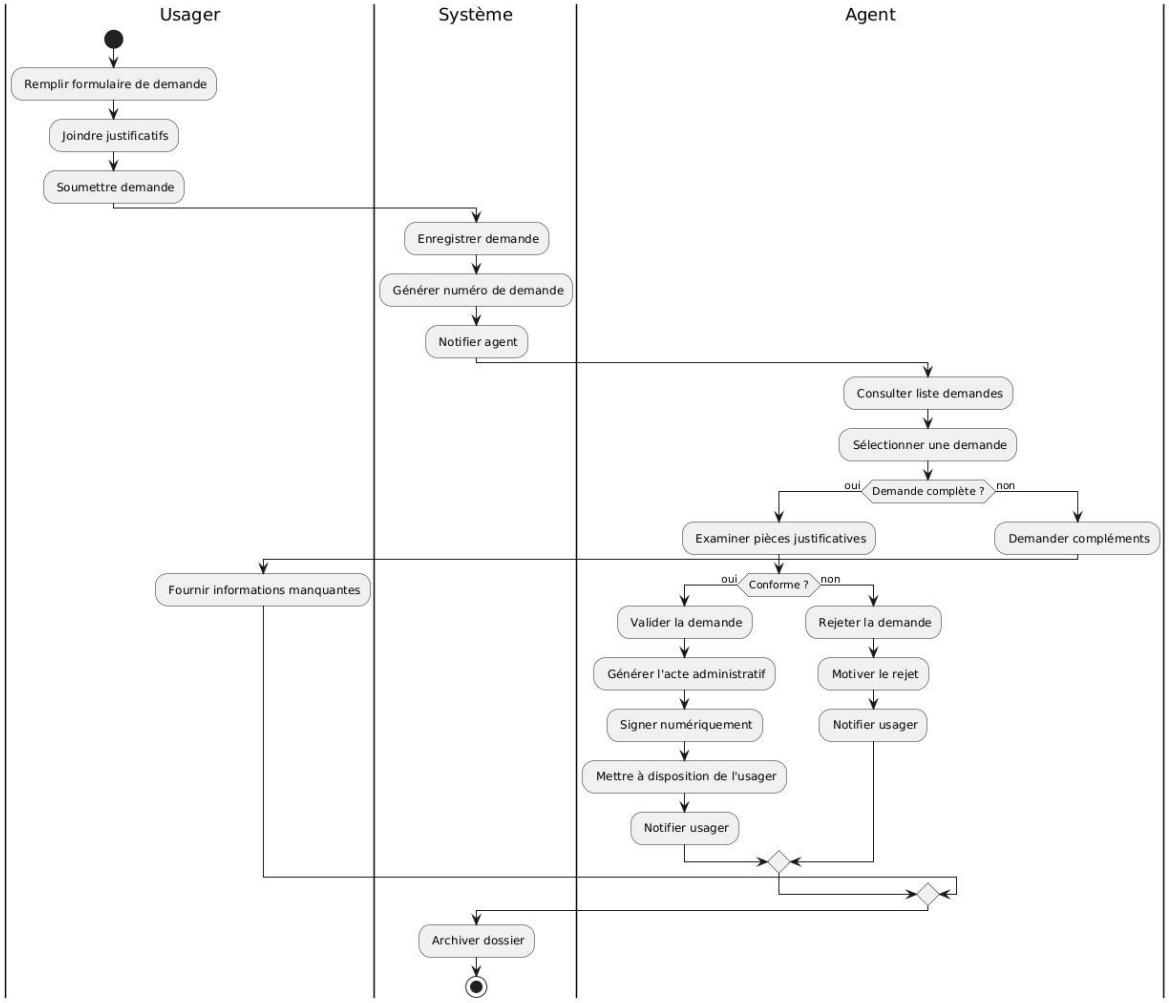

Figure 7 — Diagramme d'activités : traitement d'une demande

3. Phase 3 — Conception architecturale

3.1 Architecture du système

Le système implémente une architecture MVC étendue à 5 couches. Une couche Service est intercalée entre Controller et DAO pour isoler la logique métier des accès aux données.

Couche	Package Java	Responsabilité
Model	model / model.enums	Entités JPA persistées via Hibernate (@Entity, Lombok).
View	resources/fxml + css	Fichiers FXML (JavaFX 21). Liés aux Controllers par @FXML.
Service	service	Logique métier, règles de gestion, orchestration DAO.
Controller	controller.*	Gestion des événements JavaFX. Délègue au Service.
DAO	dao.interfaces / dao.impl / dao.utils	Accès MySQL via Hibernate SessionFactory.

Flux : FXML → Controller → Service → DAO → MySQL, puis retour inverse. Point d'entrée : Launcher.java → Main.java (javafx.application.Application).

3.2 Diagramme de composants

Le diagramme de composants représente tous les composants logiciels du projet et leurs dépendances : les vues FXML, les controllers par rôle, les services métier, les interfaces et implémentations DAO, les utilitaires et la base de données.

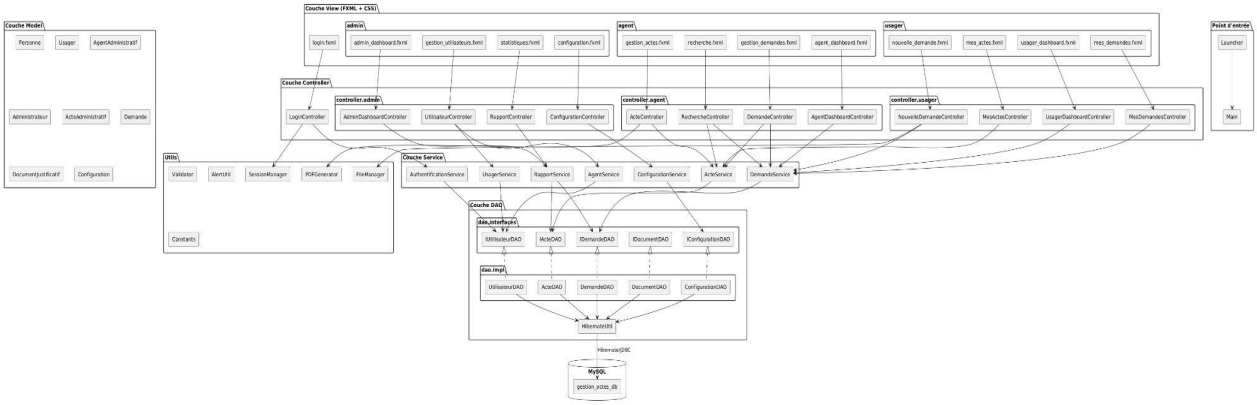

Figure 8 — Diagramme de composants

3.3 Diagramme de déploiement

Le diagramme de déploiement décrit l'environnement d'exécution physique. Chaque poste (Administrateur, Agent, Usager) embarque l'application Java avec ses vues et controllers spécifiques. Tous se connectent au serveur MySQL via Hibernate/JDBC sur le port 3306

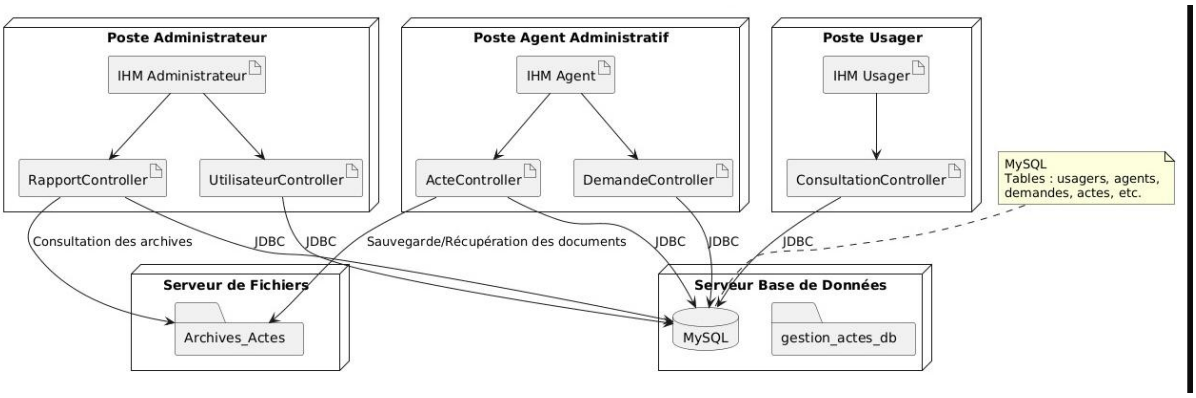

Figure 9 — Diagramme de déploiement

3.4 Organisation des packages Java

Package	Classes
(défaut)	Main.java, Launcher.java
model	Personne, Usager, AgentAdministratif, Administrateur, ActeAdministratif, Demande, DocumentJustificatif, Configuration
model.enums	TypeActe, StatutDemande, StatutActe, NiveauAcces
controller	LoginController

controller.admin	AdminDashboardController, UtilisateurController, RapportController, ConfigurationController
controller.agent	AgentDashboardController, DemandeController, ActeController, RechercheController
controller.usager	UsagerDashboardController, NouvelleDemandeController, MesDemandesController, MesActesController
service	ActeService, DemandeService, AuthentificationService, RapportService, AgentService, UsagerService, ConfigurationService
dao.interfaces	IActeDAO, IDemandeDAO, IUtilisateurDAO, IDocumentDAO, IConfigurationDAO
dao.impl	ActeDAO, DemandeDAO, UtilisateurDAO, DocumentDAO, ConfigurationDAO
dao.utils	HibernateUtil
utils	SessionManager, PDFGenerator, FileManager, Validator, AlertUtil, Constants

3.5 Interfaces DAO

Chaque DAO implémente une interface Java garantissant un faible couplage entre les couches.

Interface	Méthodes principales
IActeDAO	save(), findById(), findByNumero(), findAll(), findByUsager(), findByType(), findByStatut(), findByPeriode(), update(), delete(), countByType()
IDemandeDAO	save(), findById(), findByNumero(), findAll(), findByUsager(), findByStatut(), findByTypeActe(), findByAgentTraiteur(), update(), delete(), countByStatut()
IUtilisateurDAO	saveUsager/Agent/Admin(), findUsagerById/ByNumero(), findAgentByMatricule(), findAdminByLogin(), findAll*()/update()/delete() par type, authenticate(email, password)

IDocumentDAO	save(), findById(), findByDemande(), findAll(), update(), delete(), deleteByDemande()
IConfigurationDAO	find(), save()

3.6 Technologies utilisées

Technologie / Version	Rôle dans le projet
Java	Langage principal de développement.
JavaFX 21	Framework IHM. Vues FXML liées aux Controllers via @FXML.
Hibernate 6.3	ORM pour la persistance JPA. Génère le schéma SQL (hbm2ddl.auto=update).
MySQL 8.x	SGBD. Base gestion_actes_db sur localhost:3306.
MySQL Connector/J 8.3	Driver JDBC pour Hibernate.
Lombok 1.18	Génération automatique (@Data, @Builder, @SuperBuilder, @Slf4j).
iText 7 (8.0.2)	Génération des actes et récépissés en PDF.
SLF4J 2.0 + Logback 1.5	Journalisation des événements applicatifs.
Maven	Gestion des dépendances et build (pom.xml).

4. Plan de test

Le plan de test définit les cas de test à exécuter pour valider le bon fonctionnement du système avant sa mise en production.

4.1 Environnement de test

Composant	Valeur
OS	Windows 10/11, Linux Ubuntu 22+
JDK	OpenJDK 21
JavaFX	21
Base de données	MySQL 8.0 — gestion_actes_db
Build	Maven 3.8+ — mvn javafx:run

4.2 Comptes de test

Rôle	Login	Mot de passe
Administrateur	admin	admin
Agent administratif	agent@ifri.bj	agent123
Usager	usager@ifri.bj	(aucun)

4.3 Résumé des cas de test

Module	Nominaux	Négatifs	Total
Authentification	3	3	**6**

Espace Usager — Nouvelle demande	3	3	**6**
Espace Usager — Mes demandes	2	2	**4**
Espace Usager — Mes actes	2	2	**4**
Espace Agent — Gestion demandes	4	3	**7**
Espace Agent — Gestion actes	3	2	**5**
Espace Admin — Utilisateurs	4	3	**7**
Espace Admin — Configuration	2	2	**4**
Espace Admin — Statistiques/Rapports	5	3	**8**
TOTAL	**28**	**23**	**51**

4.4 Critères d'acceptation

Critère	Seuil d'acceptation
Taux de réussite global	≥ 90% des cas de test passés
Tests haute priorité	100% des tests haute priorité passés
Tests d'authentification	100% — critère bloquant
Exceptions non gérées	0 NullPointerException ou RuntimeException en production
Cohérence des données	Toutes les opérations CRUD vérifiées en base

5. Conclusion

Ce rapport regroupe l'ensemble des livrables des Phases 2 et 3 du projet de gestion des actes administratifs à IFRI. La modélisation UML (Phase 2) a permis de définir avec précision les fonctionnalités, les interactions entre acteurs et composants, et les flux de traitement des données.

La conception architecturale (Phase 3) a abouti à une architecture MVC étendue à 5 couches, robuste et maintenable, s'appuyant sur des technologies Java modernes (JavaFX 21, Hibernate 6.3, iText 7). L'organisation en packages et les interfaces DAO garantissent un faible couplage et une extensibilité sans remise en cause de la structure existante.

Le plan de test, avec ses 51 cas couvrant 10 modules, définit un cadre rigoureux pour la validation fonctionnelle lors de la Phase 4 (implémentation complète). Les critères d'acceptation assurent un niveau de qualité conforme aux exigences du cahier des charges.